

Herald



Protocol Research: Standard-Webhooks (outbound delivery)

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	Standard-Webhooks is the industry-led spec for webhook delivery: HMAC-SHA256 signed payload, timestamp-bound, replay-protected, with JWKS-style ephemeral signing keys (mainstreamed in 2026). Stripe, GitHub, Shopify, Linear all follow the pattern. Herald MUST adopt this for outbound notification delivery to user-configured webhook URLs (one of Herald's primary channels). Wave 4d.
Issues	open-questions: ephemeral key cadence; CloudEvents payload mandatory vs optional; max retry attempts.
Continuation	Wave 4d — open HRD-261..HRD-275 block; new module <code>commons_webhook/</code> (L1, depends on <code>commons</code> + <code>commons_storage</code>).

Constitutional anchors

- **§107 anti-bluff** — webhook tests assert: real test HTTP server (booted via `containers/`) receives the POST with correct HMAC signature; replay attack rejected; retry backoff observed.
- **§11.4.74 catalogue-check** — performed; no `vasic-digital` or `HelixDevelopment` webhook delivery module. Verdict: implement Herald-internal (the standard is simple enough — HMAC-SHA256 + timestamp; no external SDK needed).
- **§11.4.61 / branding** — tracked doc.

Table of contents

- [§1. Protocol overview](#)
- [§2. Specification deep-dive](#)
- [§3. Herald-specific applicability analysis](#)
- [§4. Step-by-step implementation guide for Herald](#)
- [§5. §107 anti-bluff testing strategy](#)

- [§6. Open questions for operator](#)
- [§7. References](#)
- [§8. Catalogue-check verdict](#)

§1. Protocol overview

Standard-Webhooks (standard-webhooks.com) is an open-source specification (Linux Foundation, late 2023) consolidating the de-facto webhook delivery patterns used by Stripe, GitHub, Shopify, Discord, Slack, Linear, and others. It standardizes: payload format, HMAC signature, replay protection, retry semantics, observability, and security best practices.

Wire format. Webhooks are HTTP POST requests:

- **URL** — receiver-configured.
- **Method** — POST always.
- **Headers** — Content-Type: `application/json` (or `application/cloudevents+json`), `Webhook-ID`, `Webhook-Timestamp`, `Webhook-Signature`.
- **Body** — JSON or CloudEvent payload.

Authentication / authenticity. HMAC-SHA256:

```
signed_payload = "{id}.{timestamp}.{body}"
signature = base64(hmac_sha256(signing_key, signed_payload))
```

`Webhook-Signature: v1,base64(sig) v1,base64(sig2)` (multi-signature for key rotation).

Replay protection. `Webhook-Timestamp` MUST be within ± 5 minutes of receiver's clock.

Delivery semantics. At-least-once delivery. Standard retry policy: exponential backoff with jitter, 5–8 attempts over 24–72 hours. Receivers MUST be idempotent (use `Webhook-ID` as idempotency key).

Ephemeral signing keys (2026 mainstreamed). Providers publish a JWKS endpoint (`/ .well-known/webhook-keys.json`) with short-lived (15min–24h) HMAC keys. Receivers cache active keys + rotate. Massive blast-radius reduction on leaked secrets.

Payload format. Standard-Webhooks recommends CloudEvents v1.0.2 as the payload (structured mode) — but allows opaque JSON.

§2. Specification deep-dive

§2.1 Headers

Header	Value	Purpose
<code>Webhook-ID</code>	UUID v4 or v7	unique event id; idempotency key on receiver
<code>Webhook-Timestamp</code>	Unix seconds	replay protection bound
<code>Webhook-Signature</code>	<code>v1,base64sig</code> <code>[v1,base64sig2 ...]</code>	HMAC-SHA256 over <code>id.timestamp.body</code>

§2.2 Signature computation

```
signed = webhook_id + "." + str(timestamp) + "." + body
mac = HMAC_SHA256(signing_secret, signed)
sig = base64(mac)
header = "v1," + sig
```

Multi-sig for key rotation: emit two v1, . . . tokens; receiver accepts if ANY matches a known key.

§2.3 Verification (receiver side)

1. Check Webhook-Timestamp is within ± 300 seconds of now.
2. Reconstruct `signed = id + "." + ts + "." + body`.
3. Compute HMAC with the stored signing secret.
4. Compare against Webhook-Signature token(s) using constant-time compare.

§2.4 Retry semantics

Standard pattern (Stripe-derived):

- Initial attempt.
- Retry 1: 1 min.
- Retry 2: 5 min.
- Retry 3: 30 min.
- Retry 4: 2 hours.
- Retry 5: 12 hours.
- Retry 6: 24 hours.
- Total window: 72 hours.

Each retry adds jitter $\pm 20\%$. Final failure $\rightarrow$ dead-letter queue.

§2.5 Observability

- Webhook send: emit OTel span `webhook.send` with `webhook.url`, `webhook.id`, `http.status_code`, `retry.attempt`.
- Failure: emit `webhook.failed` event with reason + final-attempt flag.

§2.6 Ephemeral keys (2026 pattern)

- Provider publishes JWKS at `/ .well-known/webhook-keys.json`:

```
{
  "keys": [
    { "kid": "k1", "alg": "HS256", "k": "base64-secret-1",
      "exp": 1747980000 },
    { "kid": "k2", "alg": "HS256", "k": "base64-secret-2",
      "exp": 1747993600 }
  ]
}
```

- Receiver caches active keys, refreshes when current key expires.

- Provider signs with the latest key; emits `Webhook-Signature: kid=k1,v1,sig`.

(Note: HS256 secrets in JWKS are usually wrapped in JWE in production; the JWKS endpoint is itself authenticated via mTLS or signed JWT.)

§3. Herald-specific applicability analysis

§3.1 Use case

Every Herald flavor binary needs to dispatch notifications to user-configured webhook URLs as one of N channels. Currently, Herald's `commons_messaging/channels/` has Telegram, Slack, Email, etc. — adding "webhook" as a channel:

- `commons_messaging/channels/webhook/`
- Subscriber config carries `webhook_url`, `signing_secret` (or `jwtks_url` for ephemeral).
- `pherald notify` tool POSTs to the webhook with full Standard-Webhooks compliance.

§3.2 Inbound webhooks

In addition to OUTBOUND delivery, Herald may also INGEST inbound webhooks (e.g. GitHub, Stripe, Linear → `pherald`). The same spec applies, in reverse:

- Herald exposes `/v1/webhooks/inbound/{tenant_id}/{source}` endpoints.
- Each tenant configures a `signing_secret` per source.
- Herald verifies signature + dedupes by `Webhook-ID`.

§3.3 Payload — CloudEvents

Per Wave 4c (CLOUDEVENTS.md), all Herald-emitted webhooks use CloudEvents structured-mode payloads. Wave 4d adds the HMAC layer over the structured-mode body.

§3.4 Multi-tenant

Each subscriber row has `webhook_url + webhook_secret_kid + webhook_secret`. Per-tenant, per-subscriber.

§3.5 JWT integration

Webhook delivery is NOT JWT-based — it's HMAC-based. But the SUBSCRIPTION management API (which configures the webhook URL) IS JWT-protected via `commons_auth/`.

§4. Step-by-step implementation guide for Herald

§4.1 New module `commons_webhook/`

```
commons_webhook/
├── go.mod
├── sender/
│   └── sender.go                # HMAC signer + HTTP poster
```

```

├── retry.go           # exponential backoff + jitter
├── ephemeral_keys.go # JWKS-style key rotation
├── sender_test.go
├── verifier/
│   ├── verifier.go    # for INBOUND webhooks
│   └── verifier_test.go
├── worker/
│   ├── worker.go      # pulls from webhook_deliveries table; calls
│   └── worker_test.go
└── e2e_real_webhook_test.go

```

§4.2 New Postgres table

```

CREATE TABLE webhook_deliveries (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  subscriber_id UUID NOT NULL,
  event_id UUID NOT NULL,
  webhook_url TEXT NOT NULL,
  payload JSONB NOT NULL,
  signing_kid TEXT NOT NULL,
  attempt_count INT DEFAULT 0,
  next_attempt_at TIMESTAMPTZ,
  status TEXT NOT NULL,
  -- 'pending', 'in_flight', 'delivered', 'dead'
  last_response_code INT,
  last_response_body TEXT,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX webhook_deliveries_next_attempt ON
  webhook_deliveries(next_attempt_at) WHERE status =
  'pending';

```

Migration 000006_webhook_deliveries.up.sql under commons_storage/migrations/.

§4.3 New channel adapter

commons_messaging/channels/webhook/ — Channel interface implementation for outbound webhook delivery. Enqueues a row in `webhook_deliveries`; worker picks it up.

§4.4 New worker (background)

Every flavor's `serve` runs a webhook worker goroutine: every 5 seconds, query `webhook_deliveries` where `next_attempt_at < now()` AND `status = 'pending'`; for each row, attempt POST; on failure, schedule next attempt or mark dead.

§4.5 Ephemeral key endpoint

GET /.well-known/webhook-keys.json — Herald's JWKS for HMAC. Two active keys at all times (current + rolling-to-next). Rotation every 24h (configurable).

§4.6 New e2e_bluff_hunt invariants

- **E67** — happy path: enqueue webhook delivery; test HTTP server receives POST with correct signature.
- **E68** — replay protection: same body + same timestamp → 200; +6 minutes → receiver MUST reject (test the verifier).
- **E69** — retry: test server returns 503 on first call, 200 on second; Herald retries with correct backoff.
- **E70** — dead-letter: test server returns 500 for all attempts; after 6 retries, row goes to `status = 'dead'`.
- **E71** — JWKS rotation: rotate key; new POST signed with new kid; receiver fetches JWKS; verifies.
- **E72** — INBOUND webhook: send a signed POST to Herald; assert verification + dedupe.

§4.7 HRD scaffolding

- **HRD-261** — bootstrap `commons_webhook/`.
- **HRD-262** — `webhook_deliveries` schema migration.
- **HRD-263** — sender + retry + jitter.
- **HRD-264** — ephemeral key rotation + JWKS endpoint.
- **HRD-265** — inbound verifier.
- **HRD-266** — channel adapter wiring.
- **HRD-267** — worker goroutine in every flavor's serve.
- **HRD-268** — CloudEvents payload integration (links to Wave 4c).
- **HRD-269** — `/v1/webhooks/inbound/...` ingest routes per flavor.
- **HRD-270** — e2e_bluff_hunt E67–E72.
- **HRD-271** — mutation gates.
- **HRD-272** — spec V3 §4w amendments.
- **HRD-273** — operator credentials guide: webhook secret + JWKS URL config.
- **HRD-274** — dashboards (Prometheus metrics for webhook delivery health).
- **HRD-275** — close Wave 4d.

§5. §107 anti-bluff testing strategy

The bar: a real HTTP server (booted via `containers/` — a small nginx or Go test server) receives Herald-emitted webhooks with verifiable signature; Herald correctly retries on 5xx; Herald's INBOUND endpoint verifies signed POSTs from a test sender.

§5.1 Happy-path tests

- **Test 1: outbound delivery.** Test server logs the POST; verify `Webhook-ID`, `Webhook-Timestamp`, `Webhook-Signature` headers; verify signature against known secret; verify body matches expected CloudEvent.
- **Test 2: outbound retry.** Test server returns 503 on call 1, 200 on call 2; assert `attempt_count = 2` in DB.
- **Test 3: outbound dead-letter.** Test server returns 500 always; after 6 attempts, row is dead.
- **Test 4: inbound verify.** Sender signs payload; POST to Herald; Herald accepts.
- **Test 5: inbound reject (bad sig).** Sender uses wrong key; POST to Herald; 401.
- **Test 6: replay reject.** Same `Webhook-ID` POSTed twice within 5min → second call returns 200 (idempotency) but does NOT create duplicate. Same ID after 5min still rejected for the time delta if timestamp didn't update.

- **Test 7: ephemeral key rotation.** Herald rotates key; subsequent POSTs use new kid; test server (which knows the JWKS) verifies via new kid.

§5.2 Edge cases

- Timestamp clock skew: timestamp 4 min in future → 200 (within ± 5 min window).
- Timestamp 10 min in past → receiver rejects (Webhook-Verification).
- Empty body → still verify signature (signature over empty string).

§5.3 Performance

- 1000 concurrent webhook sends → throughput target: 500 sends/s. Worker batch size tuned.

§5.4 Mutation gates

- Mutation: skip signing in `sender.go` → Test 1 sig verification FAILS.
- Mutation: skip retry backoff → Test 2 FAILS (retries too aggressively).
- Mutation: stub timestamp check → Test 6 PASSES with old timestamp (BAD; mutation proves the check is load-bearing).

§5.5 Wire-level inspection

- tcpdump the outbound POST → assert headers + body byte-for-byte.
- Manually compute HMAC offline → assert matches.

§5.6 Real-world compatibility

`scripts/e2e_webhook_stripe_clone.sh` — emit a Herald webhook formatted EXACTLY like Stripe's; run Stripe's open-source signature verifier (from `github.com/stripe/stripe-go`); assert verifier returns valid.

§6. Open questions for operator

1. **Long-lived secret OR ephemeral keys default?** Recommend: support BOTH; subscribers choose at config time. Long-lived is simpler; ephemeral is more secure.
2. **Max attempt count?** Recommend 6 (matches Stripe / GitHub).
3. **Retry window?** Recommend 72 hours.
4. **CloudEvents payload — mandatory or optional?** Recommend mandatory once Wave 4c lands.
5. **Outbound concurrency limit per tenant?** Recommend 10 concurrent in-flight per tenant; prevents accidental DDoS of a slow receiver.
6. **Should Herald publish a `"/.well-known/webhook-receiver"` descriptor** so PROVIDERS know how to send TO Herald? Wave 4d+1.
7. **Backpressure on `webhook_deliveries` table growth?** Auto-purge dead rows after 30 days.

§7. References

(All fetched 2026-05-22.)

- **Spec.** Standard Webhooks specification — <https://github.com/standard-webhooks/standard-webhooks/blob/main/spec/standard-webhooks.md>
- **Spec home.** <https://www.standardwebhooks.com/>
- **HMAC fyi.** <https://webhooks.fyi/security/hmac>
- **Stripe webhook verification.** <https://stripe.com/docs/webhooks/signatures>
- **GitHub webhook verification.** <https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries>
- **2026 best practices.** https://dev.to/young_gao/building-reliable-webhook-delivery-retries-signatures-and-failure-handling-40ff
- **Webhook delivery guarantees.** <https://codelit.io/blog/api-webhooks-delivery-guarantee>
- **Hookdeck payload best practices.** <https://hookdeck.com/outpost/guides/webhook-payload-best-practices>

§8. Catalogue-check verdict

Per §11.4.74:

- **vasic-digital:** no module.
- **HelixDevelopment:** no module.

Verdict: no-match → **implement Herald-internal in** `commons_webhook/`. The spec is simple enough (HMAC-SHA256 + timestamp + JWKS) that no external library is needed; `stdlib crypto/hmac + crypto/sha256 + net/http` suffices. No submodule needed.